

```
[ ]: -- EJERCICIO 1
```

```
[12]: a = [1,2,3]
      b = [4,5]
      c = a ++ b
      return c
      [1,2,3,4,5]
```

```
[13]: :t True
      :t "Hola mundo"
      :t [1,2,3]
```

Verdadero :: Booleano

"Hola mundo" :: String

[1,2,3] :: para todo {a}. Num a => [a]

*[14]: -- 2. ¿Qué significa que Haskell sea un lenguaje fuertemente tipado y evaluado perezosamente?
-- R// Significa que el lenguaje es extremadamente estricto con las reglas de sus datos, pero muy flexible y eficiente sobre cuándo procesa esos datos

```
[24]: doble :: Int -> Int
      doble x = 2 * x
      doble 2
      4
```

```
[25]: esMayorEdad :: Int -> Bool
      esMayorEdad edad = edad >= 18
      esMayorEdad 15
      FALSO
```

```
[34]: areaCirculo :: Float -> Float
      areaCirculo r = (r * r) * pi
      areaCirculo 5
      78.53982
```

[35]: -- ¿Qué diferencias encuentras entre los tipos Int y Float?
-- R// Que int es para valores específicos enteros y float son mas para valores inexactos y ambos tienen sus ventajas y desventajas dependiendo del problema.

```
[36]: -- EJERCICIO 2
```

```
[37]: head [10,20,30,40]
      tail [10,20,30,40]
      take 3 [1..10]
      drop 4 [1..10]
      length [1,2,3,4,5]
      10
      [20,30,40]
      [1,2,3]
      [5,6,7,8,9,10]
      5
```

[38]: -- ¿Qué hace la expresión [1..10] y cómo se relaciona con la evaluación perezosa?
-- R// Que solo toma los valores de un número n que le damos nosotros de una lista, es como definir un rango y se relaciona en que es flexible y eficiente.

```
[50]: primerElemento :: [a] -> a
      primerElemento (x:_) = x
      primerElemento [85,2,3,4]
      85
```

```
[57]: productoLista :: [Int] -> Int
      productoLista [] = 1
      productoLista (x:xs) = x * productoLista xs
      productoLista [2,3,4]
```

Utilice foldr

Encontró: Por qué no:
lista de productos [] = 1 lista de productos (x : xs) = x * lista de productos xs
productoLista xs = foldr (*) 1 xs
24

```
[61]: eliminarNegativos :: [Int] -> [Int]
      eliminarNegativos xs = filter (>= 0) xs
      eliminarNegativos [1,-2,3]
```

Eta reduce

Found: Why Not:
eliminarNegativos xs = filter (>= 0) xs
eliminarNegativos = filter (>= 0)
[1,3]

```
[67]: invertir :: [a] -> [a]
      invertir [] = []
      invertir (x:xs) = invertir xs ++ [x]
      invertir [1,2,3,4]
      [4,3,2,1]
```

```
[68]: -- EJERCICIO 3
```

```
[69]: cuadrado :: Int -> Int
cuadrado n = n * n

suma :: Int -> Int -> Int
suma a b = a + b

sumaCuadrados :: Int -> Int -> Int
sumaCuadrados x y = suma (cuadrado x) (cuadrado y)
sumaCuadrados 2 5

29
```

```
[71]: duplicar :: Int -> Int
duplicar n = n * 2

-- Forma simple
dobleListaSimple :: [Int] -> [Int]
dobleListaSimple = map duplicar

-- Forma compleja
dobleListaCompleja :: [Int] -> [Int]
dobleListaCompleja xs = map duplicar xs
dobleListaSimple [1,2,3]
```

Eta reduce

Found: **Why Not:**

```
dobleListaCompleja xs = map duplicar xs
dobleListaCompleja = map duplicar

[2,4,6]
```